

Day 11 — Robustness and evaluation

Day 10 left a working assistant with nine flows and NLU/LLM coexistence. Today you will make the transfer flow safer and turn important conversations into repeatable end-to-end tests.

You will:

1. observe the default conversation-repair patterns;
2. prevent interruptions during authentication;
3. customize chitchat;
4. validate transfer data at three different layers;
5. create an end-to-end regression suite and inspect its coverage.

Work inside this lesson's `starting-point/`. The completed project is available in `end-result/` for comparison.

1. Prepare the project

You need:

- Python 3.11 and `uv` ;
- `RASA_LICENSE` exported in your shell;
- `OPENAI_API_KEY` exported in your shell.

The Makefile supplies these demo settings to the assistant and mock bank:

```
CORE_BANKING_URL=http://localhost:8000
CORE_BANKING_TOKEN=test_token
```

From `starting-point/`, create the environments once:

```
make setup
```

Then use two terminals, both opened in `starting-point/`:

```
# Terminal 1: mock core-banking API
make run-fastapi-server
```

```
# Terminal 2: train the assistant and open Inspector
make run-rasa
```

The first training run and later end-to-end tests call the configured OpenAI model. A full e2e run makes several dozen LLM calls. With the default summary mode, each rephrased response can make two LLM calls: one to summarize the conversation history and one to rephrase the response.

Every new conversation starts unauthenticated. Use `123456` when the assistant asks for the security code. Keep the Inspector's stack pane visible while testing interruptions.

2. Observe the default repair patterns

Before editing the project, try the following conversations. The exact LLM-generated routing or wording can vary; inspect the stack and events rather than expecting identical text.

Interrupt and resume a transfer

Start a transfer, then ask for your balance:

```
user: I want to send some money to a friend
bot: For your security, please enter your one-time code.
user: 123456
bot: Who is the recipient of the transfer?
user: Wait, how much is in my checking account?
bot: Which account would you like to check?
user: checking
bot: Your checking account balance is €1250.0.
bot: Would you like to continue with transfer money?
user: yes
bot: Who is the recipient of the transfer?
```

`check_balance` is pushed above the paused transfer. After it completes, `pattern_continue_interrupted` asks whether to resume the transfer.

Trigger clarification

During a transfer, send an ambiguous card request:

```
user: Something's wrong with my card
bot: I can help, but I need more information. Which of these would you like to do:
block a card or replace a card?
```

When several flows are plausible, `pattern_clarification` presents the candidates. The candidate list may vary because the LLM chooses it.

Correct a collected value

Reach the transfer recap with an amount of `200`, then write:

```
user: Actually, make it 150 euros
bot: Ok, I am updating transfer_amount to 150.0.
bot: Please confirm: transfer €150.0 to Marco Bianchi, IBAN IT60...3456. Shall I proceed?
```

`pattern_correction` updates the slot. The recap is asked again because its collect step uses `ask_before_filling: true`.

Cancel the active flow

```
user: Forget it, cancel the whole thing
bot: Okay, stopping transfer money.
bot: Is there anything else I can help you with?
```

`pattern_cancel_flow` removes the transfer from the stack. `pattern_completed` then closes the interaction.

Stop the backend before confirmation

Reach the recap, stop Terminal 1 with Ctrl-C, and confirm the transfer:

```
bot: I couldn't reach the transfer system, so no money has moved. Please try again later.
```

This response does not come from a repair pattern. `action_execute_transfer` handles the connection error and the flow follows its failure branch. This is safer than relying on `pattern_internal_error`, which can only report a generic software failure.

Restart the mock bank before continuing:

```
make run-fastapi-server
```

3. Block interruptions during authentication

A security-code challenge should not allow digressions. Edit:

```
lumera-assistant/data/flows/shared/authenticate_user.yml
```

Replace the `security_code` collect step with:

```
- collect: security_code
  id: ask_code
  description: the one-time security code the customer provides
  ask_before_filling: true
  # While the code is being collected, ignore every command except
  # filling this slot – no digressions during a security challenge.
  force_slot_filling: true
```

`force_slot_filling: true` ignores every command except the command that fills this slot while the collect step is active.

Restart the assistant with a retrain:

```
make run-rasa
```

Ask for a balance while the assistant waits for the security code:

```
bot: For your security, please enter your one-time code.
user: Wait, how much is in my checking account?
bot: I'm sorry I am unable to understand you, could you please rephrase?
bot: For your security, please enter your one-time code.
```

The balance flow no longer starts. Use this setting only where interruptions are unsafe or meaningless.

4. Customize chitchat

The default `pattern_chitchat` refuses off-topic requests. Override it with the shipped free-chitchat response.

Create:

```
lumera-assistant/data/flows/patterns/pattern_chitchat.yml
```

```
flows:
  # Override of the shipped chitchat pattern: a flow with the reserved name
  # replaces the default (which answers utter_cannot_handle). The free-form
  # response only becomes generative once the rephraser is wired in
  # endpoints.yml – without it, the static fallback text is served.
  pattern_chitchat:
    description: handle interactions with the user that are not task-oriented
    name: pattern chitchat
    steps:
      - action: utter_free_chitchat_response
```

A flow with this reserved ID replaces the default pattern. Retrain and restart:

```
make run-rasa
```

Try an off-topic request during a transfer:

```
user: By the way, do you like pizza?
bot: Sorry, I'm not able to answer that right now.
```

This is still static fallback text. `utter_free_chitchat_response` is marked for rephrasing, but the project has no rephraser yet.

Edit `lumera-assistant/endpoints.yml`. Add this block after `action_endpoint`:

```
# Contextual Response Rephraser: rephrases responses that carry
# `metadata: rephrase: True` (our own responses don't – only the shipped
# pattern responses and utter_free_chitchat_response do). Endpoints are
# runtime configuration: changing this file needs a restart, not a retrain.
nlg:
  type: rephrase
  llm:
    model_group: openai_llm
```

`endpoints.yml` is runtime configuration, so restart the assistant without retraining. From `lumera-assistant/` in Terminal 2, run:

```
.venv/bin/rasa inspect
```

Try the pizza request again. The wording is generated and will vary. The rephraser affects every response carrying `metadata: rephrase: True`, including many default pattern responses. Keep exact, regulated messages outside this generative path.

5. Add transfer validation

Use three validation layers:

- domain validation for a reusable IBAN format rule;
- flow validation for this transfer process's €5000 limit;
- a custom action for the account balance held by the backend.

5.1 Validate the IBAN in the domain

Edit `lumera-assistant/domain/transfer_money.yml`.

Replace the `recipient_iban` slot with:

```
recipient_iban:
  type: text
  mappings:
    - type: from_llm
  # Universal format rule: an Italian IBAN starts with IT, then two check
  # digits, 27 characters in total. True wherever this slot is used, so
  # the rule lives at the domain level.
  validation:
    rejections:
      - if: not (slots.recipient_iban matches "^IT[0-9]{2}[A-Za-z0-9]{23}$")
        utter: utter_invalid_iban
```

Under `responses:`, add:

```
utter_invalid_iban:
  - text: "That doesn't look like an Italian IBAN – it should start with IT and be 27
characters long. Could you check it?"
```

The predicate rejects the candidate value. The collect step then asks for the IBAN again.

5.2 Validate the transfer limit in the flow

Edit:

```
lumera-assistant/data/flows/transfers/transfer_money.yml
```

Replace the `transfer_amount` collect step with:

```
- collect: transfer_amount
  description: the transfer amount in euros; extract only the numerical value,
ignoring any currency name or symbol
  # This process's ceiling, not the world's: the per-transfer limit
  # belongs to this flow, so it lives on this collect step.
  rejections:
    - if: slots.transfer_amount > 5000
      utter: utter_transfer_over_limit
```

In `lumera-assistant/domain/transfer_money.yml`, add this response under `responses:`:

```
utter_transfer_over_limit:
  - text: "I can't set that up: online transfers are limited to €5000 per operation.
For larger amounts, please visit a branch."
```

This rule belongs to the flow because it is a limit of this process, not a property of every numeric slot.

5.3 Validate available funds with an action

Create:

```
lumera-assistant/actions/validate_transfer_amount.py
```

```

import os
from typing import Any, Dict, List, Text

import requests
from rasa_sdk import Action, Tracker
from rasa_sdk.events import SlotSet
from rasa_sdk.executor import CollectingDispatcher

class ValidateTransferAmount(Action):
    """Layer-3 validation: the truth about available funds lives in the
    core-banking system, not in YAML. Rasa runs any action named
    validate_<slot_name> automatically whenever that slot is collected."""

    def name(self) -> Text:
        return "validate_transfer_amount"

    def run(
        self,
        dispatcher: CollectingDispatcher,
        tracker: Tracker,
        domain: Dict[Text, Any],
    ) -> List[Dict[Text, Any]]:
        amount = tracker.get_slot("transfer_amount")
        if amount is None:
            return []

        api_base = os.environ.get("CORE_BANKING_URL")
        token = os.environ.get("CORE_BANKING_TOKEN")

        try:
            response = requests.get(
                f"{api_base}/v1/balance",
                params={"account_type": "checking"},
                headers={"Authorization": f"Bearer {token}"},
                timeout=5,
            )
            response.raise_for_status()
            balance = response.json()["balance"]
        except (requests.RequestException, KeyError, TypeError, ValueError):
            # If the balance service is unreachable, don't block the customer
            # at validation: the transfer execution itself fails clean.
            return []

        if amount > balance:
            dispatcher.utter_message(
                response="utter_insufficient_funds",
                available_balance=f"{balance:.2f}",
            )
            return [SlotSet("transfer_amount", None)]

        return []

```

Rasa automatically runs an action named `validate_<slot_name>` after collecting that slot. Clearing the slot with `SlotSet("transfer_amount", None)` makes the collect step ask again.

The action fails open if the balance lookup is unavailable. The later execution action remains responsible for preventing and reporting a failed transfer.

In `lumera-assistant/domain/transfer_money.yml`, add this response under `responses:`:

```
utter_insufficient_funds:
  - text: "That's more than what's available on your checking account (€
{available_balance}), so this transfer would not go through. What amount should I use
instead?"
```

Replace the final `actions:` block with:

```
actions:
  - action_execute_transfer
  - validate_transfer_amount
```

Validate the project, then retrain and restart:

```
cd lumera-assistant
.venv/bin/rasa data validate
cd ..
make run-rasa
```

Test each rejection:

```
IBAN:  BANANA123      → rejected; the assistant asks again
IBAN:  IT60X0542811101000000123456 → accepted
Amount: 9000          → rejected by the flow limit
Amount: 2000          → rejected because the demo balance is €1250
Amount: 300           → accepted
```

YAML rejections run before the custom validation action, so inexpensive local checks happen before the backend call.

6. Build the end-to-end regression suite

Build the suite one test file at a time. Create each directory when you add its first file.

6.1 Add a shared authentication fixture

Create `lumera-assistant/e2e_tests/conftest.yml`:


```

# Fixtures visible to every test file under e2e_tests/.
# Slots listed here are pre-set right after session start, so test cases
# for sensitive flows can bypass the security-code challenge.
fixtures:
  - authenticated_user:
    - authenticated: true

```

The fixture bypasses authentication in tests that focus on another behavior.

6.2 Test repair patterns

Create `lumera-assistant/e2e_tests/patterns/cancellation.yml`:

```

test_cases:
  - test_case: cancellation_mid_transfer_pops_the_flow
    fixtures:
      - authenticated_user
    steps:
      - user: I'd like to make a transfer
        assertions:
          - flow_started:
              operator: all
              flow_ids:
                - transfer_money
      - user: Marco Bianchi
        assertions:
          - slot_was_set:
              - name: recipient_name
                value: Marco Bianchi
      - user: forget it, cancel the whole thing
        assertions:
          - flow_cancelled:
              flow_id: transfer_money

```

Create `lumera-assistant/e2e_tests/patterns/chitchat.yml`:

```

# The rephrased chitchat answer is generated fresh on every run, so this
# case asserts the flow and response action – never the exact response text.
# The action assertion also proves that our override replaced the default,
# which executes utter_cannot_handle instead.
test_cases:
  - test_case: off_topic_message_goes_to_pattern_chitchat
    steps:
      - user: do you like pizza?
        assertions:
          - flow_started:
              operator: all
              flow_ids:
                - pattern_chitchat
          - action_executed: utter_free_chitchat_response

```

The response text is generated, so the test does not assert it. Instead, it checks both the routing decision and the response action. The action distinguishes this override from the default `pattern_chitchat`, which executes `utter_cannot_handle`.

Create `lumera-assistant/e2e_tests/patterns/clarification.yml`:

```
# Which candidates the LLM lists is a judgement call that varies run to
# run (sometimes card_fees_info joins the two card flows) – and the `all`
# operator requires the exact option set. `any` pins what must hold: the
# pattern fired and offered card flows.
test_cases:
  - test_case: ambiguous_card_request_triggers_clarification
    steps:
      - user: something's wrong with my card
        assertions:
          - pattern_clarification_contains:
              operator: any
              flow_ids:
                - block_card
                - replace_card
```

`operator: any` allows the LLM to include other plausible candidates while still checking that card flows were offered.

Create `lumera-assistant/e2e_tests/patterns/correction.yml`:

```

# Correction at the recap: the amount must end at the corrected value.
# validate_transfer_amount is stubbed empty ("validation passes") – stub
# runs must stub every custom action the case touches.
stub_custom_actions:
  validate_transfer_amount:
    events: []
    responses: []

test_cases:
- test_case: correction_at_the_recap_updates_the_amount
  fixtures:
    - authenticated_user
  steps:
    - user: I want to send money to Marco Bianchi
      assertions:
        - flow_started:
            operator: all
            flow_ids:
              - transfer_money
    - user: IT60X0542811101000000123456
      assertions:
        - bot_uttered:
            utter_name: utter_ask_transfer_amount
    - user: "200"
      assertions:
        - bot_uttered:
            utter_name: utter_ask_transfer_confirmed
    - user: actually, make it 150
      assertions:
        - flow_started:
            operator: all
            flow_ids:
              - pattern_correction
        - slot_was_set:
            - name: transfer_amount
              value: 150.0

```

Create `lumera-assistant/e2e_tests/patterns/nlu_trigger_coexistence.yml`:

```

# Coexistence guard: a high-confidence balance message must keep starting
# check_balance (via the nlu_trigger or the LLM – the assertion holds for
# either routing path). The balance fetch is stubbed.
stub_custom_actions:
  action_fetch_balance:
    events:
      - event: slot
        name: current_balance
        value: 1250.0
      - event: slot
        name: balance_fetch_ok
        value: true
    responses: []

test_cases:
  - test_case: high_confidence_balance_message_starts_check_balance
    steps:
      - user: what's my balance
        assertions:
          - flow_started:
              operator: all
              flow_ids:
                - check_balance
      - user: checking
        assertions:
          - bot_uttered:
              utter_name: utter_current_balance

```

6.3 Test the transfer path

Create `lumera-assistant/e2e_tests/transfers/transfer_happy_path.yml`:

```

# The transfer happy path, with the core-banking call stubbed: this file
# tests the conversation logic, not the backend – it must pass with the
# mock API down.
#
# This case uses the step-script format (user turns + expected responses
# and slots) rather than per-turn assertions: it freezes the whole
# journey, and only checks our own literal responses – never rephrased
# pattern text.
stub_custom_actions:
  action_execute_transfer:
    events:
      - event: slot
        name: transfer_id
        value: TRF-0042
      - event: slot
        name: transfer_ok
        value: true
    responses: []
  # Stub runs must stub EVERY custom action the case touches – including
  # validate_transfer_amount, which the collect machinery calls
  # automatically. An empty stub means "validation passes".
  validate_transfer_amount:
    events: []
    responses: []

test_cases:
  - test_case: transfer_happy_path_completes
    fixtures:
      - authenticated_user
    steps:
      - user: I want to send money to Marco Bianchi
      - slot_was_set:
        - recipient_name: Marco Bianchi
      - utter: utter_ask_recipient_iban
      - user: the IBAN is IT60X0542811101000000123456
      - utter: utter_ask_transfer_amount
      - user: "100"
      - slot_was_set:
        - transfer_amount: 100.0
      - utter: utter_ask_transfer_confirmed
      - user: yes, go ahead
      - utter: utter_transfer_done
      - user: no thanks
      - utter: utter_transfer_wrap_up

```

The stubs keep the test focused on conversation behavior. A stubbed test file must stub every custom action its cases call, including automatically invoked validation actions.

6.4 Test input validation

Create `lumera-assistant/e2e_tests/validation/transfer_input_validation.yml`:

```
# The two YAML validation layers, each rejected once and then accepted.
# The layer-3 validate_transfer_amount action is stubbed empty here
# ("validation passes") – its insufficient-funds catch needs the
# core-banking API up, so it is exercised live against the mock instead.
```

stub_custom_actions:

validate_transfer_amount:

events: []

responses: []

test_cases:

- **test_case:** garbage_iban_is_rejected_then_valid_iban_accepted

fixtures:

- authenticated_user

steps:

- **user:** I want to send money to Marco Bianchi

assertions:

- **flow_started:**

operator: all

flow_ids:

- transfer_money

- **user:** the IBAN is BANANA123

assertions:

- **bot_uttered:**

utter_name: utter_invalid_iban

- **bot_uttered:**

utter_name: utter_ask_recipient_iban

- **slot_was_not_set:**

name: recipient_iban

value: BANANA123

- **user:** sorry, IT60X0542811101000000123456

assertions:

- **slot_was_set:**

name: recipient_iban

value: IT60X0542811101000000123456

- **test_case:** amount_over_the_transfer_limit_is_rejected

fixtures:

- authenticated_user

steps:

- **user:** I want to send money to Marco Bianchi

assertions:

- **flow_started:**

operator: all

flow_ids:

- transfer_money

- **user:** IT60X0542811101000000123456

assertions:

- **bot_uttered:**

utter_name: utter_ask_transfer_amount

- **user:** "9000"

assertions:

- **bot_uttered:**

utter_name: utter_transfer_over_limit

```
- user: ok, 900 then
  assertions:
    - slot_was_set:
      - name: transfer_amount
        value: 900.0
```

The insufficient-funds behavior is not included here: stubbing its rejection would only test the stub. Test that integration manually with the mock bank running, as in section 5.

6.5 Add the test commands

Append these targets to `starting-point/Makefile`:

```
# Run the regression suite against the latest trained model.
# Stubbed custom actions are a beta feature, hence the env flag. Every turn
# still goes through the real LLM – a full run costs a few dozen calls.
run-e2e-tests:
  cd lumera-assistant && RASA_PRO_BETA_STUB_CUSTOM_ACTION=true .venv/bin/rasa test
e2e e2e_tests/

# Same suite, plus the flow/step coverage report.
run-e2e-coverage:
  cd lumera-assistant && RASA_PRO_BETA_STUB_CUSTOM_ACTION=true .venv/bin/rasa test
e2e e2e_tests/ --coverage-report
```

The beta flag enables custom-action stubs. User turns still go through the real command generator.

Run the suite from `starting-point/`:

```
make run-e2e-tests
```

LLM routing is nondeterministic. A failure can represent a real routing variation rather than a syntax error; inspect the failed assertion before changing the test.

Then generate coverage:

```
make run-e2e-coverage
```

Coverage identifies untested flows and branches. The authentication flow remains uncovered because the shared fixture bypasses it; the wider portfolio and transfer failure branches also need later cases. Treat this report as a concrete list of missing tests.

7. Review the result

You have added:

- interruption protection for the security-code collect step;
- a custom chitchat pattern and contextual rephrasing;

- domain-, flow-, and backend-level transfer validation;
- regression tests for repair patterns, coexistence, validation, and the transfer path;
- commands for running the suite and its coverage report.

Compare your `starting-point/` with `end-result/`. Apart from local runtime artifacts and harmless ordering differences within YAML mappings, both projects should now contain the same configuration, actions, and tests.